

TFL CLIENT

ESPECIFICACION MAESTRA DE PRODUCTO Y DESARROLLO

Version 1.0 | Septiembre 2026

Definicion central

TFL Client sera un launcher de Minecraft moderno y ligero para cualquier jugador, desarrollado por TFLives, con identidad propia y capaz de subsistir totalmente sin TFL Network. TFL Network podra aparecer como colaboracion natural dentro del ecosistema, nunca como dependencia del producto.

Referencia tecnica: la eficiencia y madurez de CubicLauncher.
Referencia visual/UX: la calidad percibida del launcher de LabyMod.
Objetivo: una identidad propia de TFLives, no una copia de ninguno.

Documento de entrega para desarrollo

Este documento define la fuente de verdad inicial del proyecto. Las decisiones tecnicas pueden evolucionar si existe una justificacion medible de rendimiento, mantenimiento o compatibilidad.

1. Resumen ejecutivo

TFL Client es un launcher generalista de Minecraft con identidad de TFLives. Debe servir tanto a un jugador que nunca ha entrado a TFL Network como a un miembro de la comunidad. Su propuesta no es encerrar al usuario en un ecosistema, sino ofrecer una experiencia moderna, rapida, visualmente cuidada y suficientemente completa para instalar, organizar y jugar Minecraft.

Principio rector

El launcher debe sentirse como un producto independiente de TFLives, no como un panel de acceso a TFL Network. La relacion con la network sera promocional y contextual: presencia del servidor, informacion puntual y futuras colaboraciones, sin romper el uso generalista.

1.1 Objetivos principales

- Arranque rapido, uso de memoria contenido y bajo impacto mientras Minecraft esta abierto.
- Experiencia visual moderna, propia y coherente en Dark y Light.
- Acceso obligatorio mediante una cuenta de Minecraft oficial o una cuenta cracked/offline.
- Gestion completa de instancias, loaders y contenido sin obligar al usuario a manipular carpetas.
- Modrinth y CurseForge como proveedores oficiales de contenido.
- Soporte de Vanilla, Forge, Fabric, NeoForge y Quilt.
- Actualizaciones del launcher automaticas con aviso al usuario.
- Funcionamiento util sin Internet para todo lo que ya este instalado localmente.
- Arquitectura preparada para Linux y macOS aunque Windows 10+ sea la prioridad inicial.
- Base mantenible por un equipo muy pequeno, evitando infraestructura innecesaria.

1.2 No objetivos iniciales

- Convertir TFL Client en una aplicacion exclusiva para TFL Network.
- Crear una red social, chat propio o sistema comunitario complejo en la primera version.
- Crear una tienda, marketplace propio o sistema de monetizacion.
- Mantener un marketplace de temas de usuario en la primera etapa.
- Construir servicios remotos propios cuando una solucion local o estandar sea suficiente.
- Reescribir sistemas maduros solo por preferencia estetica del codigo.

2. Identidad del producto

Elemento	Definicion
Nombre	TFL Client
Desarrollador / marca	TFLives
Categoria	Launcher y gestor de instancias de Minecraft
Publico	Cualquier jugador de Minecraft; no requiere pertenecer a TFL Network
Relacion con TFL Network	Producto hermano. Colaboraciones y promocion, sin dependencia funcional
Plataforma prioritaria	Windows 10 y Windows 11
Plataformas futuras	Linux y macOS, facilitadas por decisiones multiplataforma desde el inicio
Idiomas de lanzamiento	Espanol e ingles
Referencia de eficiencia	CubicLauncher y otros launchers ligeros que el desarrollador considere tecnicamente superiores

Elemento	Definicion
Referencia visual	Launcher de LabyMod como inspiracion de calidad, jerarquia y acabado; no copiar su UI

2.1 Personalidad visual

La interfaz debe combinar un lenguaje de Minecraft moderno con una presentacion premium y de escritorio. Se permite glassmorphism moderado, siempre que no comprometa legibilidad ni rendimiento. La referencia de LabyMod debe entenderse como calidad percibida: composicion clara, controles con buen acabado, motion intencional y sensacion de producto terminado.

- Nada de aspecto de dashboard SaaS generico.
- Nada de sobrecargar cada pantalla con tarjetas, bordes y texto auxiliar.
- Usar profundidad, blur y transparencias con moderacion y segun perfil de calidad.
- Priorizar superficies y jerarquia visual antes que decoracion.
- Dark y Light son experiencias oficiales; ninguna debe sentirse como tema secundario roto.
- No se requiere sistema de temas instalables por usuarios en la primera version.

3. Estrategia tecnica y decision de baseline

CubicLauncher ya demostro ser una referencia util por rendimiento, madurez y organizacion; sin embargo, TFL Client no queda obligado a ser un fork permanente. Antes de fijar el baseline definitivo, el desarrollador debe hacer una evaluacion corta y practica de las alternativas.

Opcion	Cuando elegirla	Riesgo principal
Fork/adaptacion de CubicLauncher	Si conserva gran parte de las funciones deseadas, compila de forma estable y permite rebrand sin deuda excesiva.	Heredar decisiones/servicios/licencia/UX que no encajen.
Base propia sobre Tauri + Rust	Si el coste de desacoplar o modificar Cubic se acerca al coste de construir una base mas limpia.	Reimplementar demasiado y tardar en alcanzar madurez.
Otra base open source	Si una alternativa supera claramente a Cubic en arquitectura, licencia, mantenimiento y cobertura funcional.	Cambiar de base por novedad sin beneficio medible.

Regla de decision

La investigacion inicial debe ser corta. No se busca el launcher perfecto del mundo: se busca una base que reduzca trabajo real. La opcion elegida debe justificarse por rendimiento, funcionalidades ya resueltas, mantenibilidad, licencia y facilidad de rebrand.

3.1 Stack recomendado

Capa	Tecnologia recomendada	Razon
Desktop shell	Tauri 2	Binario pequeno, menor memoria que Electron y acceso nativo mediante Rust.
Core nativo	Rust	Procesos, filesystem, descargas, hashing, Java, launch y concurrencia con alto control de recursos.
Frontend	Svelte 5 + SvelteKit	UI reactiva ligera; encaja con Cubic y reduce coste de mantener una capa web compleja.

Capa	Tecnologia recomendada	Razon
Lenguaje UI	TypeScript	Tipado y contratos claros entre UI y servicios.
Build UI	Vite	Desarrollo rapido y tooling maduro.
Package manager	Bun	Rapido y ya probado en la base de Cubic; no es runtime obligatorio de produccion.
Web runtime Windows	WebView2 mediante Tauri	Reutiliza componentes del sistema en lugar de empaquetar Chromium completo.
IPC	Tauri commands + events	UI delgada; el trabajo pesado permanece en Rust.
Updater	Tauri Updater firmado	Actualizaciones seguras y controlables sin construir un actualizador desde cero.
Distribucion	Tauri Bundler / NSIS en Windows	Instalador nativo, versionado y soporte de updater.
Repositorios de contenido	Modrinth + CurseForge	Cobertura amplia de mods, modpacks, resource packs y shaders.

3.2 Regla de rendimiento

Rust hace el trabajo pesado. La interfaz presenta, coordina y reacciona.

Descargas, verificacion de hashes, descompresion, gestion de archivos, deteccion de Java, preparacion del classpath, lanzamiento de Minecraft, importacion de packs y tareas costosas no deben ejecutarse en el hilo principal de la interfaz.

4. Arquitectura objetivo

```

Svelte UI
|
v
Typed frontend services
|
v
Tauri Commands / Events
|
v
Rust application services
|
+-- Accounts
+-- Instances
+-- Java runtimes
+-- Minecraft versions/loaders
+-- Content providers
+-- Importers
+-- Launch/process manager
+-- Updater/telemetry
|
v
Filesystem / Network / OS processes

```

4.1 Principios de arquitectura

- La UI no debe contener logica profunda de Minecraft.
- Los comandos Tauri deben ser adaptadores finos; la logica reutilizable vive en servicios Rust.
- Los modelos intercambiados por IPC deben estar tipados y ser pequenos.

- Las tareas largas deben informar progreso mediante eventos, no bloquear la ventana.
- Los proveedores Modrinth y CurseForge deben compartir interfaces comunes donde sea razonable.
- La arquitectura debe evitar rutas o APIs exclusivas de Windows en el core salvo adaptadores de plataforma.
- No crear microservicios locales ni bases de datos pesadas si archivos estructurados/SQLite ligero resuelven el problema.
- No duplicar caches ni sistemas de descarga para cada pantalla.

5. Acceso, cuentas y onboarding

TFL Client debe requerir una identidad antes de entrar a la experiencia principal. El usuario no entra primero al launcher para luego descubrir que necesita configurar una cuenta.

5.1 Flujo de entrada

1. Primera ejecucion: pantalla de bienvenida y seleccion/inicio de cuenta.
2. Opciones obligatorias: Minecraft oficial (Microsoft) y cuenta cracked/offline.
3. Tras autenticar o crear la cuenta offline, se entra al launcher.
4. Debe existir gestion posterior de multiples cuentas, cambio de cuenta y cierre de sesion.
5. Una cuenta cracked/offline es una identidad valida y permanente del launcher, no un modo invitado.

5.2 Operacion sin conexion

El launcher debe funcionar sin Internet para operaciones locales: abrir la aplicacion, ver instancias, cambiar ajustes, gestionar archivos locales y lanzar una instancia que no necesite descargar nada nuevo. Las funciones de red deben fallar de forma clara y no impedir el uso local.

6. Instancias y Minecraft

La instancia es la unidad principal del producto. Debe representar un entorno de Minecraft autocontenido, entendible y gestionable.

Area	Requisito
Version	Seleccion de versiones de Minecraft compatibles con el loader elegido.
Loaders	Vanilla, Forge, Fabric, NeoForge y Quilt como soporte oficial.
Java	Deteccion automatica y posibilidad de runtime administrado por el launcher.
Memoria	RAM configurable por instancia con valores recomendados razonables.
Contenido	Mods, modpacks, resource packs, shader packs y datapacks.
Archivos	Acceso controlado a carpeta de instancia, logs, screenshots, worlds y configs cuando corresponda.
Acciones	Jugar, reparar/verificar, duplicar, editar, eliminar y exportar si la arquitectura final lo admite.
Estado	Ultimo uso, version, loader y errores relevantes visibles sin saturar la tarjeta.

6.1 Workspace de instancia

Aunque CubicLauncher sirve de referencia, TFL Client puede presentar una instancia de forma mas completa. La meta es que el usuario no necesite navegar por carpetas para las tareas normales. El workspace debe agrupar funciones por intencion y mantener una jerarquia clara, evitando paneles densos innecesarios.

7. Contenido y modpacks

7.1 Proveedores oficiales

- Modrinth
- CurseForge

Ambos deben considerarse proveedores de primera clase. Ninguno debe quedar oculto como integracion secundaria. La UI puede unificar busqueda y gestion, siempre respetando los terminos y capacidades de cada API.

7.2 Tipos de contenido

- Mods
- Modpacks
- Resource packs
- Shader packs
- Datapacks

7.3 Importacion

La importacion oficial se centra en paquetes/modpacks portables, no en migrar instalaciones enteras de otros launchers.

- Modrinth `.mrpack`.
- Packs ZIP compatibles con CurseForge.
- Otros formatos de modpack razonables si una especificacion estable y mantenible lo justifica.

No es requisito inicial importar directamente perfiles internos de Prism, MultiMC, Lunar, Badlion u otros launchers.

8. Rendimiento y perfiles de calidad

TFL Client debe poder permanecer abierto mientras Minecraft esta ejecutandose sin convertirse en una carga importante. El rendimiento no se define solo por FPS de la interfaz: incluye memoria, CPU en reposo, cantidad de procesos, actividad de disco y trabajo de red.

Perfil	Objetivo
Lite	Minimo movimiento, transparencias/blur reducidos o desactivados, fondos simples y minima actividad en segundo plano.
Balanced	Experiencia recomendada: buen acabado visual con coste moderado.
Experience	Maxima calidad visual permitida por la implementacion, sin eliminar funcionalidad en hardware modesto.

El usuario decide el perfil. No se debe prohibir Experience en hardware debil; como maximo se puede recomendar otro perfil. Cuando la ventana este minimizada u oculta, animaciones y render innecesario deben pausarse.

8.1 Metas practicas

- No ejecutar polling agresivo de filesystem o red cuando existen eventos o intervalos mas razonables.
- No mantener animaciones a alta frecuencia si la ventana no es visible.
- Lazy-load de pantallas y recursos pesados cuando aporte una mejora medible.
- Liberar previews 3D, imagenes grandes y recursos temporales cuando dejan de utilizarse.
- Descargas concurrentes con limites; nunca saturar disco/red sin control.
- Evitar dependencias frontend grandes por efectos que puedan resolverse con CSS o codigo existente.

9. UX y estructura de informacion

La distribucion exacta de CubicLauncher no se considera definitiva. El programador puede proponer una organizacion mejor, pero debe evitar cambiar informacion de sitio sin una ganancia clara. La prioridad es que las acciones frecuentes aparezcan primero.

9.1 Superficies principales sugeridas

Superficie	Proposito
Inicio	Instancia principal/reciente, continuar jugando, noticias o estado minimo y acceso promocional a TFL Network.
Biblioteca / Instancias	Ver, ordenar, crear y administrar instancias.
Descubrir	Buscar contenido/modpacks en Modrinth y CurseForge.
Cuenta	Identidad actual, cambio/gestion de cuentas y skin cuando proceda.
Ajustes	Launcher, Java, descargas, idioma, apariencia Dark/Light y perfil de calidad.

9.2 TFL Network dentro del launcher

En la primera etapa, TFL Network debe aparecer como servidor promocional obligatorio. Puede mostrarse en Inicio, lista de servidores o una zona contextual apropiada. No debe bloquear el uso ni convertir el launcher en una aplicacion exclusiva de la network.

Independencia obligatoria

Si TFL Network esta caida o deja de existir, TFL Client debe seguir funcionando como launcher general de Minecraft.

10. Apariencia y personalizacion

- Dark y Light oficiales.
- Selector de calidad visual: Lite, Balanced y Experience.
- No se necesita marketplace/sistema de temas instalables en la primera version.
- No se requiere personalizacion de tipografia, fondos arbitrarios ni editor visual complejo.
- El estilo debe ser coherente con TFLives sin reciclar automaticamente la UI de la web, Discord o tienda.

11. Discord Rich Presence

La Rich Presence es una caracteristica importante de identidad y debe sentirse especialmente trabajada. Debe reflejar el contexto sin revelar informacion sensible.

Estado	Ejemplo de presencia
Inicio	Explorando TFL Client
Biblioteca	Gestionando sus instancias
Instancia	Preparando Minecraft 1.xx - Fabric
Lanzando	Iniciando una instancia
Singleplayer	Jugando en un mundo local
Multiplayer menu	Buscando un servidor

Estado	Ejemplo de presencia
Servidor	Jugando en <servidor> cuando el dato sea publico y seguro
Descubrir	Explorando mods y modpacks

La presencia debe permitir controles de privacidad. No publicar IPs privadas, nombres de mundos locales, rutas, nombres de cuenta offline ni informacion que el usuario razonablemente espere mantener privada.

12. Telemetria, privacidad y diagnostico

TFL Client puede recopilar datos anonimos orientados a mejorar el producto, pero debe hacerlo con una politica simple y transparente. El proyecto no necesita una plataforma de analitica empresarial compleja.

12.1 Datos utiles permitidos

- Version del launcher y sistema operativo.
- Arquitectura de CPU y clase general de hardware, sin identificadores unicos.
- Loader/version de Minecraft utilizada de forma agregada.
- Duracion de arranque y errores tecnicos anonimizados.
- Perfil visual seleccionado.
- Uso agregado de funciones para priorizar mantenimiento.

12.2 Datos que no deben recopilarse

- Tokens Microsoft o credenciales.
- Contraseñas.
- Contenido de chats.
- Rutas personales completas.
- Lista completa de archivos privados.
- IPs privadas.
- Identificadores persistentes innecesarios para seguir al usuario.

Se recomienda explicar la telemetria en onboarding/ajustes y permitir desactivarla. Crash reporting puede ser parte del mismo consentimiento.

13. Actualizaciones

La experiencia objetivo es similar a Discord: el launcher detecta actualizaciones automaticamente, informa al usuario y prepara la instalacion sin exigir que busque manualmente un instalador.

- Comprobar actualizaciones en segundo plano con frecuencia razonable.
- Descargar solo desde canales/release endpoints controlados.
- Verificar firma antes de instalar.
- Avisar cuando la actualizacion esta lista.
- Aplicar al reiniciar/cerrar o mediante accion explicita segura.
- Nunca actualizar mientras pueda corromper una instalacion o proceso de Minecraft activo.

14. Plataformas y compatibilidad

Plataforma	Estado
Windows 10 x64	Soporte oficial inicial obligatorio.
Windows 11 x64	Soporte oficial inicial obligatorio.
Windows ARM64	Arquitectura futura; evitar decisiones que la bloqueen.

Plataforma	Estado
Linux x64	Preparado arquitectonicamente; soporte oficial posterior.
Linux ARM64	Futuro si tooling/dependencias lo permiten.
macOS Apple Silicon	Futuro; considerar firma/notarizacion y paths desde el diseño.
macOS Intel	No prioridad, sujeto a necesidad real.

El core debe usar abstracciones multiplataforma para paths, apertura de URLs, procesos y almacenamiento. No hardcodear `C:\` ni separadores de Windows fuera de adaptadores específicos.

15. Seguridad

- Tokens de cuentas oficiales almacenados mediante mecanismos seguros del sistema/Tauri; nunca texto plano si existe alternativa razonable.
- Descargas verificadas por hash cuando el proveedor lo facilita.
- HTTPS para APIs y updater.
- Updater firmado.
- Validacion de rutas al extraer ZIP/modpacks para evitar path traversal.
- No ejecutar scripts arbitrarios incluidos en modpacks.
- No ampliar permisos Tauri o CSP sin necesidad.
- Logs no deben imprimir tokens ni secretos.

16. Datos locales y estructura

Si se parte de CubicLauncher, los datos nuevos de TFL Client no deben vivir indefinidamente bajo `.cubic`. Se recomienda una raíz propia como `.tflclient` o el directorio de datos de aplicacion nativo de cada SO. Los datos existentes de Cubic no deben eliminarse ni moverse automaticamente.

La estructura exacta puede variar, pero debe separar claramente configuracion, cache, instancias, runtimes Java, logs y assets.

17. Flujo de desarrollo y calidad

El proyecto sera desarrollado por un equipo muy pequeno y no remunerado inicialmente. El proceso debe proteger la calidad sin convertir cada cambio en burocracia.

17.1 Definicion minima de terminado

- La funcionalidad hace lo solicitado de extremo a extremo.
- La aplicacion compila.
- El frontend no introduce errores de type-check relevantes.
- El core Rust pasa `cargo check` o validacion equivalente.
- Se prueba la ventana nativa en Windows cuando el cambio afecta runtime/IPC/UI.
- No rompe una funcion existente conocida.
- Los errores introducidos por el cambio se corrigen antes de darlo por terminado.

17.2 IA y vibe coding

El uso de IA esta permitido y no se considera un problema. La autoria importa menos que el resultado. El desarrollador sigue siendo responsable de entender suficientemente el cambio para mantenerlo, detectar regresiones y corregirlo cuando falle.

Regla practica

Codigo generado por IA es aceptable si integra bien, compila, no duplica sistemas existentes y el desarrollador puede

repararlo. No se exige una metodologia anti-IA.

18. Git, versionado y upstream

- Usar Git desde el inicio y commits legibles.
- Si se adopta un fork, conservar `upstream` para consultar fixes del proyecto original.
- No hacer merges ciegos de upstream; revisar cambios.
- Separar cambios de identidad, infraestructura y funcionalidad cuando sea razonable.
- No guardar `node\_modules`, `target`, caches, instancias, tokens ni builds locales en Git.

El esquema final de versiones puede definirse al estabilizar la primera release. Si se parte de un upstream con version alta, evitar cambiar artificialmente a `1.0.0` hasta decidir como se comportara el updater.

19. Roadmap recomendado

Fase	Resultado
0 - Discovery tecnica	Elegir baseline: Cubic, otra base o implementacion propia. Justificacion corta y prueba de compilacion.
1 - Foundation	Identidad TFL Client, cuentas obligatorias, storage propio, updater seguro, arquitectura estable.
2 - Minecraft core	Instalacion/lanzamiento robusto de Vanilla + loaders + Java.
3 - Instancias	Creacion, edicion, workspace, logs, mundos/configs y acciones clave.
4 - Contenido	Modrinth + CurseForge, mods/modpacks/resourcepacks/shaders/datapacks e importacion de packs.
5 - UX visual	Aplicar identidad final Dark/Light y perfiles Lite/Balanced/Experience con calidad tipo LabyMod.
6 - Ecosistema	Servidor promocional TFL Network, Rich Presence avanzada y telemetria anonima.
7 - Release	Updater, instalador Windows, QA de regresion, privacidad/licencia y primera version publica.

20. Evaluacion tecnica inicial para el programador

Antes de reescribir o adoptar CubicLauncher, realizar una comparacion concreta. El objetivo no es producir un informe academico, sino tomar una decision.

Criterio	Preguntas
Rendimiento	¿Cuanta RAM/CPU consume en reposo y navegando? ¿Puede permanecer abierto con Minecraft?
Cobertura	¿Cuántas funciones obligatorias ya resuelve correctamente?
Arquitectura	¿El core puede evolucionar sin que la UI controle logica de Minecraft?
Licencia	¿La licencia es compatible con la forma en que TFLives distribuira el producto?

Criterio	Preguntas
Rebrand	¿Marca, endpoints y updater pueden desacoplarse sin tocar medio proyecto?
Mantenimiento	¿Dependencias activas, código entendible, builds reproducibles?
Multiplataforma	¿La base facilita Linux/macOS o contiene acoplamientos fuertes a Windows?
UX	¿Conviene mantener, reorganizar o reemplazar la distribución actual?

Salida esperada de Discovery

Una decisión escrita de pocas páginas o un README técnico: baseline elegido, por qué, riesgos conocidos y primer milestone. Después se empieza a construir.

21. Criterios de aceptación de la primera release

- Se instala y abre en Windows 10/11 x64.
- Obliga a seleccionar/iniciar una cuenta oficial o cracked antes del shell principal.
- Permite crear y lanzar Vanilla, Forge, Fabric, NeoForge y Quilt.
- Permite administrar instancias sin necesidad de editar manualmente archivos para tareas normales.
- Integra Modrinth y CurseForge.
- Gestiona mods, modpacks, resource packs, shaders y datapacks.
- Importa al menos `.mrpack` y packs ZIP compatibles con CurseForge.
- Funciona offline para instalaciones ya disponibles localmente.
- Dark y Light están completos.
- Lite, Balanced y Experience tienen diferencias reales de coste visual.
- Updater detecta y aplica releases firmadas con aviso.
- TFL Network aparece como servidor promocional sin convertirse en dependencia.
- Rich Presence refleja contexto de manera útil y privada.
- No existen errores de branding ajeno en superficies de usuario.
- No se almacenan secretos en texto plano cuando existe una alternativa segura.

22. Decisiones explícitamente delegadas al desarrollador

No todo debe quedar congelado en una especificación. El desarrollador puede tomar decisiones técnicas siempre que respeten los objetivos.

- Elegir CubicLauncher, otra base o una implementación propia tras Discovery.
- Organización interna exacta de crates/módulos.
- Formato de persistencia local cuando sea mantenible y portable.
- Bibliotecas concretas de HTTP, serialización, logging y async dentro del ecosistema Rust.
- Distribución exacta de información en pantallas, respetando los principios UX del documento.
- Técnica concreta para blur, fondos y animaciones por perfil visual.
- Estrategia de cache y concurrencia, siempre que priorice estabilidad y rendimiento.

23. Restricciones contra sobreingeniería

- No crear backend web central solo para que el launcher funcione.
- No crear cuentas TFL obligatorias.
- No crear sistema de plugins del launcher en v1 salvo necesidad real ya demostrada.
- No introducir base de datos remota para configuraciones que son locales.
- No duplicar APIs de Modrinth/CurseForge con proxy propio sin una razón operativa.

- No implementar una red social.
- No implementar marketplace de temas.
- No mantener dos motores distintos para la misma operacion por compatibilidad imaginaria.
- No sacrificar semanas de trabajo para ahorrar cantidades irrelevantes de RAM.

24. Como medir si TFL Client esta saliendo bien

Dimension	Indicador practico
Rendimiento	Launcher usable en hardware modesto y sin carga de CPU significativa en reposo.
Estabilidad	Crear/actualizar/lanzar instancias repetidamente sin corrupcion o estados inconsistentes.
Claridad	Usuario nuevo entiende como instalar Minecraft/modpack sin guia externa.
Mantenimiento	Un unico desarrollador puede localizar y corregir un fallo sin reescribir subsistemas.
Identidad	Se reconoce TFL Client sin parecer un skin de Cubic o copia de LabyMod.
Independencia	TFL Network o servicios promocionales pueden fallar sin impedir jugar.

25. Brief final para el programador

Construye un launcher de Minecraft moderno, ligero y generalista desarrollado por TFLives. Debe combinar la eficiencia que demuestran proyectos como CubicLauncher con el nivel de acabado visual que inspira LabyMod, pero terminar con una identidad propia. Windows 10/11 es la plataforma inicial, Tauri + Rust + Svelte es la direccion tecnica preferida, y cualquier desviacion debe justificarse por una mejora real.

El usuario debe iniciar con una cuenta oficial o cracked, crear y administrar instancias con los principales loaders, obtener contenido desde Modrinth y CurseForge, importar modpacks, jugar offline cuando todo este instalado y recibir actualizaciones del launcher de forma segura. La UI ofrece Dark/Light y perfiles Lite/Balanced/Experience. TFL Network aparece como colaboracion promocional, nunca como dependencia.

La prioridad no es acumular funciones: es lograr un producto coherente, rapido, estable, visualmente excelente y mantenible por un equipo pequeño.

26. Insumos que se definiran durante implementacion

- Logo e icono final de TFL Client en formatos de produccion.
- Paleta visual final y especificacion del tema Light/Dark.
- Discord Application ID y assets para Rich Presence.
- Destino final de releases y clave publica del updater.
- Politica de privacidad/telemetria publicada.
- URL definitiva de soporte/repositorio cuando el proyecto se haga publico.

27. Estado de esta especificacion

Campo	Valor
Version	1.0
Fecha	11 de septiembre de 2026
Propietario de producto	TFLives
Estado	Base aprobada para iniciar Discovery y desarrollo
Prioridad	Windows 10/11 x64; arquitectura preparada para Linux/macOS
Documento vivo	Si. Cambios de alcance importantes deben actualizar esta especificacion.

Nota: las referencias a CubicLauncher y LabyMod son referencias de calidad tecnica/visual. No implican copiar codigo, assets, marcas o interfaces fuera de lo permitido por sus respectivas licencias y derechos.